

P2.9b — partial-failure semantics

Project: network-visualiser **Status:** agreed. Option B agreed 22 September 2026; the section 8 sub-decisions and the section 4 run policy agreed 23 September 2026. Ready to implement. **Written against:** HEAD `56fcf7a` **Scope:** what the shared build function does when classification fails partway through a build, and what every consumer shows.

1. Decision

A build that loses some of its classifications still returns the **full network**.

- People whose classification did not complete get a **fourth display state**, `not_classified`, shown as **"Not classified yet"**, together with the reason and a **Retry** control.
- This state is never counted as, rendered as, or described as "needs review". That is the D-42 rule: a failed call is not an answer.
- The build function reports this explicitly, so the app, the API and the MCP server all get the same honest result.

Rejected alternatives:

- **A, all or nothing.** It shows nothing when 430 of 442 people are fine.
- **C, a threshold that switches between A and B.** It adds a cut-off that would need justifying, and gives the user no clearer answer than B.

2. Facts from the code that make B possible

These were checked against HEAD, not assumed.

- **The layout does not depend on classification.**
- `projection_3d.py` runs UMAP on the embeddings alone (`run_umap_3d` , L72–81).
- The graph's edges are cosine similarity over the same embeddings (`graph.build_graph`). `cluster` is carried on each node as an attribute and never used to build edges.
- So every person can be placed on the canvas, and linked, without a single classifier answer.
- **The current chain still creates an ordering dependency.** `projection_3d.load_inputs` (L51–69) reads `clustered.csv`, but only for `person_index` alignment. As a result, a failed classification run blocks the layout today, even though the maths doesn't need it. The build function removes this: the layout stages run first and never read classifier output.
- **The cache is already safe.**
- D-42 made failures raise (`ClassificationFailed`), never pad.
- Cache writes sit in a `finally` block (`claude_classifier.classify_claude` , L281–322), so answers that did succeed are kept.
- A retry therefore pays only for the titles that failed.
- **What is missing is the per-item outcome.** `classify_claude` raises on the first failure and returns nothing for the rest of the run. `classify_pipeline.classify_people` (L57) has no path for "this title failed" at all.

3. The build contract

The shared build function (name to be settled when it is written) returns a `BuildResult`:

Field	Meaning
<code>nodes</code>	The full node table. One row per person in the export, all with coordinates.
<code>complete</code>	<code>True</code> only if every person with a title has a classifier answer (assigned or abstained).
<code>unclassified</code>	For each <code>person_index</code> that isn't classified: its reason (section 4). Empty when <code>complete</code> .
<code>provenance</code>	<code>model</code> , <code>prompt_version</code> and <code>source</code> , as already specified for P2.9b.

Rules:

- **Bad input raises before any spend.** A malformed or empty export, or the wrong columns, raises `InvalidExport` , and no API call is made. That is an error, not a partial result.
- **A partial result is a success with a flag, never an exception.** For the API (item 6), `complete: false` is a normal 200 response carrying `unclassified`. Only `InvalidExport` and failures of the build itself are errors. This belongs in the written API contract.
- **An empty title is still an abstention**, exactly as today (`classify_people` , L66–72). The model was never asked, so there is no call that could have failed.

4. Failure reasons and what a run does next

Reason	Cause	What the run does
<code>service_busy</code>	Rate limit, overload, a timeout or a connection error, still failing after the SDK's own retries	Stops. Every title not yet answered gets this reason.
<code>key_failed</code>	Authentication or permission error, or insufficient credit	Stops. Every title not yet answered gets this reason. Normally this can't happen, because the key is validated before the build; this covers a key that dies mid-run.
<code>budget_reached</code>	The free tier's per-session limit or global daily budget ran out mid-build	Stops. Every title not yet answered gets this reason.
<code>unanswerable</code>	<code>ClassificationFailed</code> : one title, alone in its own call, still had no usable answer	Continues. Only that title is marked.

Why a service error stops the run instead of trying the next batch: once the SDK's retries have been used up, further batches are very likely to fail the same way. Continuing costs time and gives the user nothing Retry wouldn't give them. The rule is also simple: any API error except `ClassificationFailed` stops the classification stage.

A failure is its own type. It is never a `Classification` with `tier="abstain"`. `classify_claude` gains a non-raising variant that returns answers and failures separately. The CLI keeps the raising behaviour.

5. The fourth state

Base table

- A new column, `classification_status`, takes one of three values: `assigned`, `abstained` or `not_classified`.
- A second new column, `unclassified_reason`, is empty unless the status is `not_classified`.
- `is_uncertain` keeps exactly its current meaning: the model declined. It is `True` only for `abstained`.
- `soc_major` on a `not_classified` row is a named sentinel, `NOT_CLASSIFIED`, and **never "99"**. "99" means the model abstained, and conflating the two is exactly the D-42 shape.

Display precedence in `state._row_state`:

correction > `not_classified` > `needs_review` > `classified`

Wording

Place	Text
Header	442 people · **128** need review · **12** not classified yet with a <i>Retry 12</i> button
Reasons, shown under the header	<code>service_busy</code> : "the classification service was busy" <code>key_failed</code> : "your API key was rejected or ran out of credit" (user's key), or "the classification service was unavailable" (project key) <code>budget_reached</code> : "today's free allowance ran out" <code>unanswerable</code> : "no usable answer for this title"
Profile card	Label "Not classified yet". Provenance line "Classification didn't complete: ". No confidence line.
Summaries and chat captions	A separate clause: "12 are not classified yet". Never folded into "need review".

Retry

- Retry sends only the titles in the `not_classified` set, then reassembles the table.
- It never re-embeds or re-projects, so **coordinates are byte-identical before and after a retry**. That is a test (section 9).
- When the reason is `budget_reached`, Retry is replaced by the key field and the demo, as the free-tier decision specifies.
- When the reason is `key_failed` and the key was the user's own, Retry is replaced by the key field.

The review baseline M follows the base table. A retry can turn some not-classified people into abstentions, which enlarges the backlog. `review_counts` already recomputes M from the base it is given, so M changes on a retry, and only then. That is the honest behaviour, and it needs its own test.

6. Where the code changes

File	Site at 56fcf7a	Change
<code>src/claude_classifier.py</code>	<code>classify_claude</code> L281	Non-raising variant returning answers and failures separately; exceptions mapped to the reasons in section 4
<code>src/classify_pipeline.py</code>	<code>classify_people</code> L57	Status and reason columns; <code>NOT_CLASSIFIED</code> sentinel
<code>src/dashboard/state.py</code>	<code>constants</code> L64-66, <code>row_state</code> L95, <code>apply_corrections</code> L112	<code>STATE_NOT_CLASSIFIED</code> ; an <code>ALL_STATES</code> tuple; precedence as in section 5
<code>src/dashboard/summaries.py</code>	<code>coverage_counts</code> L240, <code>coverage_sentence</code> L273	<code>not_classified</code> field and clause
<code>src/assistant.py</code>	<code>build_payload</code> L267, <code>SYSTEM_PROMPT</code> L310, <code>resolve_people</code> L435, <code>coverage_counts</code> L480, <code>coverage_sentence</code> L491	New payload flag; a prompt rule saying these people have no group and were not judged uncertain; <code>PersonRef</code> flag; caption clause; see D-53 below
<code>src/dashboard/canvas.py</code>	<code>_point_colour</code> L146	Distinct marker for the new state: an open circle, so it reads as "empty", not as another grey
<code>src/dashboard/card.py</code>	<code>group_label</code> L93, <code>provenance_label</code> L111, <code>confidence_note</code> L129, <code>render</code> L216-229	Label, provenance line, no confidence line
<code>src/dashboard/people_list.py</code>	<code>queue_frame</code> L120	Unchanged by design (it selects <code>needs_review</code> only), with a test pinning that
<code>app.py</code>	<code>counter</code> L167-169	Clause and Retry control
<code>src/dashboard/chat.py</code>	<code>table_fingerprint</code>	No change: a retry changes <code>display_state</code> , which already clears a stale highlight
<code>tools/check_uncertainty_honesty.py</code>	local state constants L45-47	Import the constants from <code>state.py</code> instead of copying them; new checks (section 9)
<code>tools/sweep_count_surfaces.py</code> , Gate D equivalence test	—	Re-run for the new state; any new count-bearing string is enumerated and gets a verdict

7. Found while writing this record

Both are logged in the defect register alongside this record. Neither is live on the current data. **Both turn into wrong output the moment a fourth state exists.**

D-53: a second implementation of the coverage count. `assistant.py` defines its own `coverage_counts` (L480). It isn't a call to `summaries.coverage_counts` (L240). Gate D's record lists "4× `coverage_counts`" as a single source, but it is two functions with the same name. It agrees today because both encode the same three states. Adding a fourth to one and not the other is the D-35 shape.

Agreed: fixed before the fourth state lands, as its own commit. `assistant.coverage_sentence` reads `summaries.coverage_counts`, and the duplicate is deleted.

D-54: silent fall-throughs on an unrecognised state. Three places:

- `canvas._point_colour` falls through to the needs-review grey.
- `card.group_label` falls through to "Needs review" when the name is blank.
- `centrality.build_canonical_table` fills a missing `is_uncertain` with `False` (L143), which makes a person absent from `classified.csv` show as *classified*.

Each would quietly render a new or missing state as an old one.

Fix: every branch on `display_state` becomes exhaustive, and an unrecognised state raises. The `centrality` fill goes away with the in-memory build, which joins with `validate="one_to_one"` instead.

Also stale: `graph.py` L47 describes `cluster_labels.csv` as a "tracked input". `.gitignore` L36 ignores it, which is why a clean deploy can't run the current chain. The build function removes the file dependency.

8. Sub-decisions (agreed 23 September 2026)

1. **Can the user correct a not-classified person by hand?** *Agreed: yes.* A correction is a human verdict on a title, and it doesn't matter whether the model's call failed. It outranks every base state, as it already does. It removes that person from the Retry set, and doesn't touch the review baseline M.
2. **What does a session export store?** *Agreed: nothing new.* "Not classified" is a property of the build, not of the user's work. Corrections made on such people save and restore by name and title like any other.
3. **How does the chat describe these people?** - *Agreed:* a payload flag, a prompt rule that says they have no group and were **not** judged uncertain, and a caption clause. - The model may include them when their title is relevant, marked as not classified yet. - Never inside a group total, and never as "unreviewed".

9. Definition of done for the implementation

The build function

- A fake client that fails at batch *k* with a service error:
- the build still returns every person with coordinates;
- `complete` is `False`;
- batch *k* and everything after it are `not_classified` with reason `service_busy`;
- nothing that failed is cached.
- An auth error mid-run: the run stops, the call count proves no further batches were sent, and the remaining titles get `key_failed`.
- A `ClassificationFailed` on one title: only that title is marked, and the other batches still run.
- A retry sends exactly the failed titles, which the fake client's recorded calls prove. After a successful retry `complete` is `True`, and the coordinates are unchanged.
- Bad input raises `InvalidExport` with zero calls made.

State and surfaces

- For every state in `ALL_STATES`, each consumer handles it explicitly: canvas, card, both caption functions, the payload, and `queue_frame`. An unrecognised state raises. This is the D-54 test.
- `not_classified` people never appear in any needs-review count, on any surface. This is checked on the same table as the D-33 equivalence test.
- A correction on a `not_classified` person makes them `classified` and removes them from the Retry set.
- M changes after a retry that adds abstentions, and at no other time.

Harnesses

- `check_uncertainty_honesty` gains three checks:
- `not_classified ∩ needs_review = ∅`;
- no group appears on the card or in the payload for `not_classified`;
- a correction resolves it.
- `check_chat_grounding` gains a case where the model is shown not-classified people and must not describe them as uncertain or give them a group.
- `sweep_count_surfaces` finds no new unexempted "N of M" strings.

Gate D re-run

- The count-surface integrity checks are repeated for the new state, in one continuous session, with a written verdict per surface.

10. Not in scope here

- Key validation, the free-tier counters, provenance display, session lifetime and embeddings cache keying. Each is its own P2.9b item. This record only fixes how their failures surface.
- The API's full contract. Section 3 states the one clause it must carry.